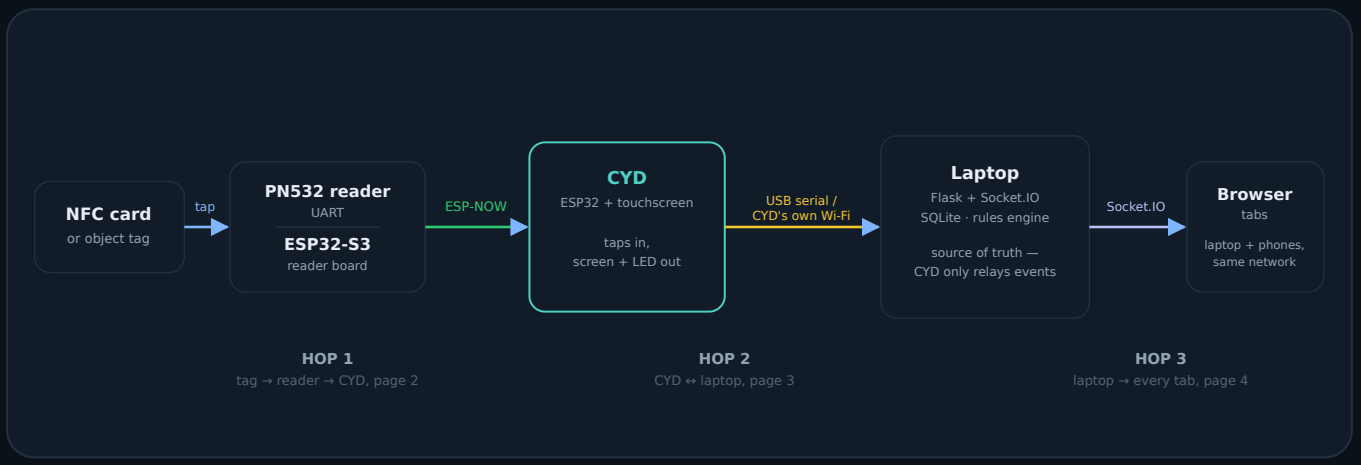
How It Talks to Itself

NFC card/tag → ESP32-S3 reader → CYD touchscreen → laptop → every open browser tab.

Three hops carry a tap on a plastic card all the way to a colored status on screen. The first hop is a short-range wireless relay between two small boards. The second hop is the CYD talking to the laptop, over a wire or over its own Wi-Fi. The third hop is the laptop pushing the result out live — nothing on this page is ever polled for.



Why two ESP32 boards, not one?

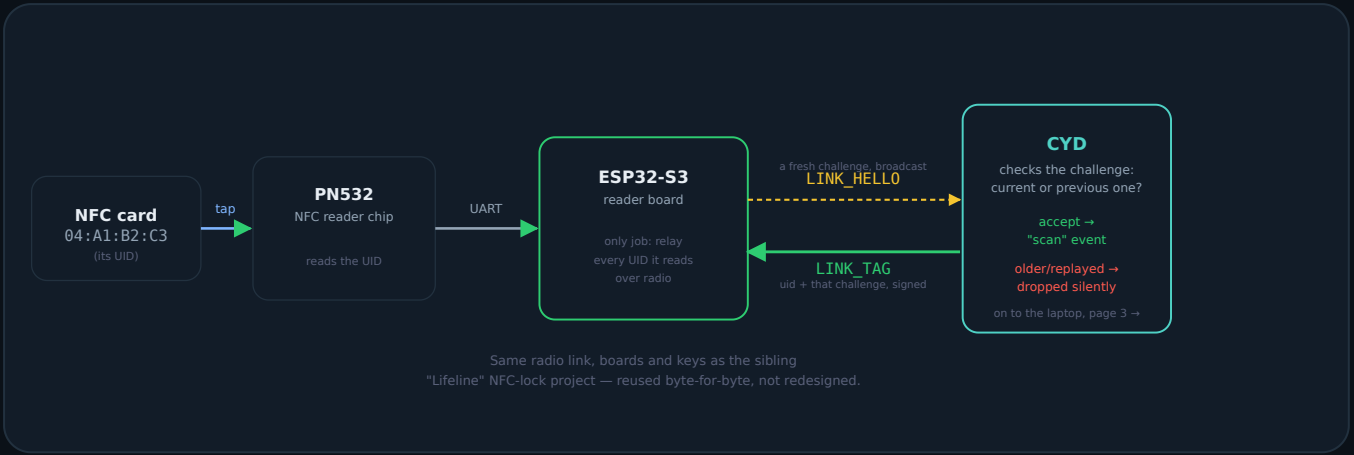
The PN532 NFC chip is already wired to its own small ESP32-S3 board from an earlier project. Reusing that working board and its radio link was simpler and safer than rewiring an NFC reader onto the CYD and writing new reader code from scratch.

Why does the CYD talk two ways to the laptop?

USB is the simple default. But a cable is a liability mid-demo — a trip, a tug, a loose port. The CYD can also run off a bare power bank and reach the laptop over its own Wi-Fi, as a deliberate fallback.

1 Tag to CYD: the wireless relay

NFC card/tag → PN532 → ESP32-S3 reader board → **ESP-NOW** → CYD



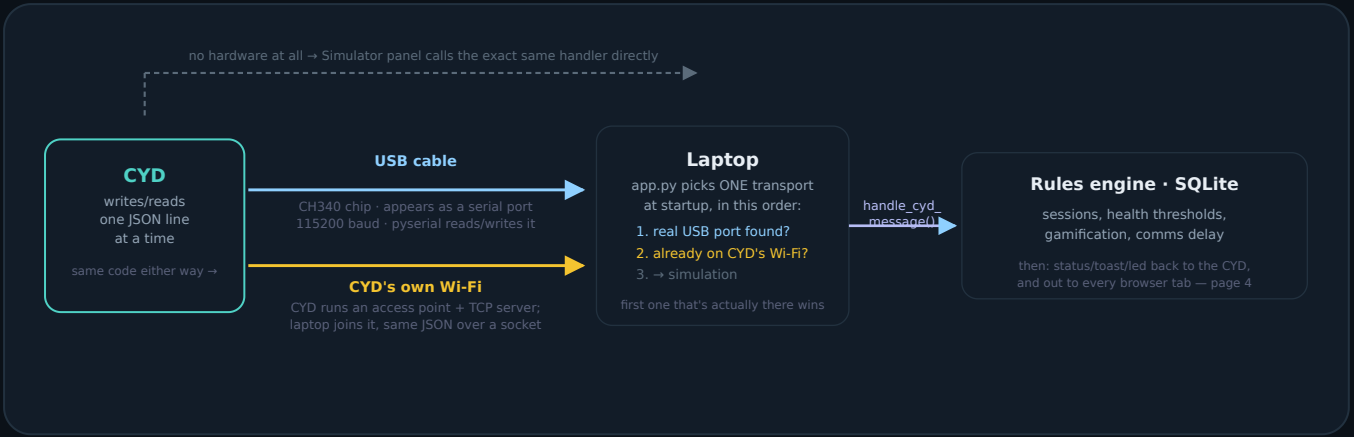
The reader board's whole job is to watch the PN532 for a UID and, the instant it sees one, hand it to the CYD over **ESP-NOW** — Wi-Fi radio hardware used as a direct, router-free link between exactly two boards. It is encrypted and cryptographically signed, so a third radio can't read or forge a tap.

Replay protection works like a login nonce: the CYD keeps broadcasting a small random `LINK_HELLO` challenge. Every `LINK_TAG` the reader sends must carry the current (or the immediately previous) challenge value. Record a real tap and play it back later, and its challenge is stale — the CYD just ignores it.

> **Not a simulator UID.** A real card's UID looks like `04:A1:B2:C3` (colon-hex). The Simulator panel's fake taps (`A1000001`, `T2000001`…) are plain demo strings typed on the laptop — they skip this whole wireless hop and go straight into the same code the CYD's real events reach, which is what makes the simulator a safe stand-in if the hardware misbehaves mid-demo.

2 CYD to laptop: one language, two wires

Every message, either direction, is one line of JSON text. Only the transport underneath changes.



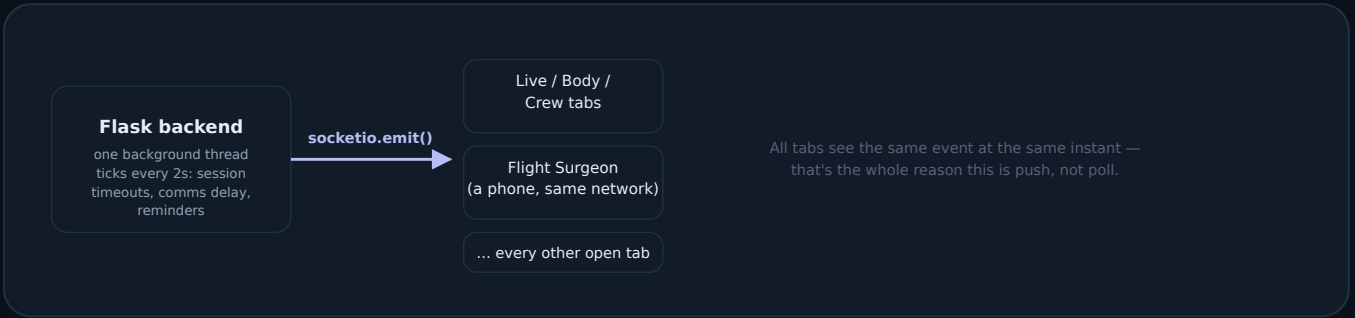
Whichever wire it travels over, a message is the same one line of JSON either way. A tap arrives as `{"type":"scan","uid":"A1000002"}`; the laptop might answer with `{"type":"session","name":"Ahmad","level":"GREEN"}` to open that astronaut's menu on screen, or `{"type":"led","color":"YELLOW"}` to set the RGB light. Nothing about that JSON changes between the two transports — only how the bytes get there does.

```
CYD → laptop   {"type":"scan","uid":"A1000002"}
laptop → CYD   {"type":"session","name":"Ahmad","level":"GREEN","points":120}
laptop → CYD   {"type":"led","color":"YELLOW"}
```

> **No flags needed.** Plug in a USB cable, or join the CYD's `AstroHealthCYD` Wi-Fi network, or do neither — `app.py` tries them in that order at startup and just uses whichever is genuinely there. A real USB cable always wins over Wi-Fi if both happen to be available at once.

3 Laptop to every screen: pushed, not polled

The moment anything changes, the server announces it. No browser tab ever asks "anything new?".



Every write — a new reading, a session opening, an alert firing — goes through SQLite and then straight out through `socketio.emit()` to every open tab on the laptop and any phone on the same network. There's no refresh button and no interval timer anywhere in the browser code; a tab just sits there until the server tells it something changed.

Quick reference

HOP	CARRIES	SECURED BY	FALLS BACK TO
`Tag → S3 → CYD`	a UID, nothing else	ESP-NOW encryption + a challenge/nonce against replay	— (no fallback; it's always this link on real hardware)
`CYD ↔ laptop`	one JSON object per line	USB (physical cable) or the CYD's own password-protected Wi-Fi	Simulation mode — same handler, called directly from the UI
`Laptop → browsers`	whatever just changed	same local network / localhost	— (a tab just shows stale data until the next event)

> **The laptop is always the source of truth.** The CYD never decides who's allowed to do anything or what a reading means — it only relays raw events up and shows back whatever the laptop tells it to display. That's deliberate: it keeps all the real logic in one place that's easy to change without reflashing a board.